

IBM Bob Work Queue and Journal Plan

Concrete tasks to assign Bob, expected outputs, acceptance checks, and evidence for the submission

Internal build window July 21-31, 2026	Submission deadline Friday, July 31 at 11:59 PM ET
--	--

Working title: CreatorLens - an AI creative director for short-form video.

Bob should be used across planning, implementation, testing, debugging, documentation, and deployment. The journal is not busywork: it is the evidence that Bob was the primary development tool.

1. Repository Documentation Layout

Use one root **README.md** for the required public overview, then keep detailed records in normal Markdown files under **docs/**. Markdown files can have any filename ending in **.md**.

```
creatorlens/  
README.md  
docs/  
BOB_JOURNAL.md  
ARCHITECTURE.md  
API_CONTRACT.md  
DECISIONS.md  
frontend/  
backend/  
sample_media/  
.env.example
```

Add a link in README.md such as: [IBM Bob Development Journal](docs/BOB_JOURNAL.md).

2. Standard Bob Task Format

Every Bob request should include five things so the output is useful and reviewable:

- **Context:** what CreatorLens does and which repository files matter.
- **Task:** one bounded deliverable, not "build the whole app."
- **Constraints:** stack, schema, supported media length, security, and scope.
- **Acceptance test:** exact behavior that proves the task is done.
- **Output request:** files to modify, tests to add, and a summary of decisions.

Reusable prompt wrapper

You are working in the CreatorLens repository, an AI creative director for 30-second to 3-minute videos. IBM Bob is our primary development tool.

Task: [ONE SPECIFIC TASK]
Relevant files: [FILES OR DIRECTORIES]
Constraints: [STACK, SCHEMA, SCOPE, SECURITY]
Acceptance criteria: [TESTABLE CONDITIONS]

Before editing, inspect the relevant code and state your plan. Make the smallest coherent change, add or update tests, explain any assumptions, and list the commands I should run to verify the result. Do not silently change public API contracts or add unrelated features.

3. Bob Task Queue by Day

Date	Task to assign Bob	Expected output and acceptance check
Jul 21	Convert the MVP into user stories, architecture, risks, repo layout, and issue backlog. Review the plan against submission requirements and judging criteria.	docs/ARCHITECTURE.md draft; prioritized issues; explicit scope cut list; no feature beyond the 10-day plan.
Jul 21	Scaffold the repository with frontend, backend, docs, .env.example, formatting, and test commands.	Fresh clone starts both skeleton services; README has temporary run instructions.
Jul 22	Implement or review the FastAPI project/upload skeleton and Pydantic schemas.	Mock upload and analysis response pass automated tests and are consumable by frontend.
Jul 22	Implement or review the React/Next.js upload form and mock results page.	User can select a file, enter brief fields, submit, and see mock scorecards/feedback.
Jul 23	Help implement media metadata, audio extraction, scene/silence metrics, and normalized evidence JSON.	One fixture creates deterministic evidence; timestamps obey video duration.

Date	Task to assign Bob	Expected output and acceptance check
Jul 24	Design and review the Granite/watsonx prompt, JSON schema, validation, retry, and model-error handling.	Three fixtures produce valid responses; malformed output is caught without crashing.
Jul 25	Debug frontend/backend integration, CORS, serialization, progress states, and timestamp navigation.	Live upload shows real results and markers seek the player correctly.
Jul 26	Implement or review one FFmpeg automatic edit and its API contract.	Approved edit creates a playable revised preview and reports progress/errors.
Jul 27	Perform a repository-wide integration review focused on hard-coded paths, secrets, missing dependencies, race conditions, and broken setup steps.	Issue list is resolved or prioritized; fresh-clone happy path passes.
Jul 28	Review the product against all five judging criteria and recommend only high-impact polish.	Action list directly tied to technical execution, innovation, fit, feasibility, and impact.
Jul 29	Draft and critique the public README sections, architecture explanation, Bob usage section, and troubleshooting steps.	README contains every required section and accurately matches the working code.
Jul 30	Create a demo-video script under 3 minutes and a final submission checklist. Review public-facing wording for unsupported claims.	Timed script fits <=3 minutes; backup demo path and required links are listed.
Jul 31	Run a final release audit: tests, secrets, public links, install steps, certificates, and submission page.	No critical blocker remains; release tag/commit is identified.

4. Detailed Bob Assignments

Architecture critic

Ask Bob to identify hidden coupling, unnecessary components, missing interfaces, and the smallest deployable architecture. Require a recommendation, not automatic code changes.

Issue decomposer

Give Bob one feature and require tasks that each fit into a few hours, with dependencies and acceptance tests.

Code implementer

Ask Bob to inspect existing files first, change a bounded set of files, add tests, and preserve API contracts.

Debugger

Provide the exact error, reproduction command, expected behavior, relevant logs, and files. Ask for ranked hypotheses before edits.

Test author

Ask Bob for unit, contract, and one end-to-end test using small fixtures. Require deterministic tests that do not call paid APIs by default.

Security reviewer

Ask Bob to inspect for committed keys, unsafe uploads, path traversal, arbitrary shell execution, oversized files, and sensitive logging.

Documentation writer

Ask Bob to draft from the actual code and commands, then manually verify every claim before committing.

Demo coach

Ask Bob to map each demo moment to a judging criterion and remove anything that does not fit within three minutes.

5. Bob Journal Entry Template

```
## 2026-07-21 - Architecture and MVP scope

**Team member:** Name
**Goal:** What we needed to accomplish
**Bob task:** The concise instruction given to Bob
**Files/context provided:** Files Bob inspected or changed
**Bob contribution:** Plan, code, tests, debugging, review, or documentation
**Human decisions:** What we accepted, rejected, or modified and why
**Verification:** Commands/tests run and result
**Related work:** Issue, commit, or pull request link
**Next step:** The immediate follow-up
```

A useful entry can be 6-12 lines. Record meaningful development work, not every tiny autocomplete. Add entries at the end of each working session while the details are fresh.

6. Journal Quality Checklist

- ☐ Entry has a date and team member.
- ☐ The goal and Bob task are specific.
- ☐ The entry distinguishes Bob output from human decisions.

- [] Verification or test evidence is included.
- [] A commit, issue, pull request, screenshot, or artifact is linked when available.
- [] No API keys, private prompts containing secrets, or personal data are copied into the journal.
- [] The README links to the journal.

7. Tasks Bob Must Not Be Allowed to Do Unchecked

- Change public API schemas without notifying both frontend and backend owners.
- Add large frameworks or dependencies merely because they are familiar.
- Claim an IBM service or model is integrated when the code uses a mock or another provider.
- Commit credentials, local machine paths, copyrighted sample media, or large generated videos.
- Rewrite a working subsystem during the integration freeze.
- Generate unsupported performance, impact, accuracy, or time-savings claims for the README or demo.
- Replace human review of AI recommendations; CreatorLens should remain a collaborative assistant.

8. Final Bob Evidence to Show Judges

Evidence	What it proves
README "How IBM Bob Was Used" section	Bob was central to planning, code, tests, debugging, and documentation.
docs/BOB_JOURNAL.md	Dated, concrete development history with human oversight.
Issues/commits/PRs referenced from journal	Bob-assisted work became actual implementation rather than hypothetical prompts.
Demo mention and architecture slide	The team can explain Bob's role clearly and quickly.
Working prototype	Bob usage resulted in a real, integrated product.